

Pin, Timer, UART ve Hertz

STM32 üzerinde bir pinin ne olduğu, timer'ın nasıl saydığı, iki cihazın UART ile nasıl konuştuğu ve bütün bunları birbirine bağlayan saat frekansı.

01 · Pinler

Alternate function, GPIO modları, pull-up/pull-down.

02 · Timer'lar

Sayaç, modlar, PWM ve duty cycle.

03 · UART

TX/RX, baud rate, 8N1 veri çerçevesi.

04 · Hertz

Frekans, periyot, PSC/ARR ve PLL.

01 Pinler gerçekte nedir?

Bir STM32'de yüzlerce pin vardır, ama hepsi aynı değildir. Pinler çoklu fonksiyon içeren yapılardır: aynı pin hem GPIO çıkışı, hem UART TX, hem timer PWM çıkışı olabilir. Hangi rolü üstleneceği yazılım tarafından seçilir.

Pin adı iki parçadan oluşur: port harfi ve pin numarası. PA0 port A'nın 0. pini, PB4 port B'nin 4. pini, PC13 port C'nin 13. pinidir.

Aynı pin bir anda yalnızca **bir** fonksiyonda çalışır. Seçim CubeMX'te yapılır.



- Dijital çıkış — LED yak
- Dijital giriş — buton oku
- Analog okuma — potansiyometre

- GPIO → UART_TX / UART_RX
- GPIO → Timer PWM çıkışı
- ADC → Buzzer / motor kontrol

- USART
- TIM
- TIM

ŞEKİL 1 — Tek bir pinin alabileceği roller (alternate function).

ALTERNATE FUNCTION (AF)

Hangi pinin hangi çevre birimi için kullanılacağını CubeMX ayarlar. UART kullanmak istiyorsanız o pinleri USART moduna çevirmeniz gerekir; GPIO olarak bırakılırsa UART çalışmaz. Pin doğru, mod yanlışsa hiçbir hata mesajı almazsınız — yalnızca veri gelmez.

1.1 Ana pin grupları

Fonksiyon	Açıklama	Örnek pinler
GPIO	Herhangi bir dijital giriş/çıkış	PA4, PB4, PB5
UART / USART	Seri iletişim (TX, RX)	PA2 (TX), PA3 (RX)
ADC	Analog okuma (0–3,3 V)	PA0, PA1
Timer / PWM	Zamanlama veya hız kontrolü	PA6, PA7
I2C	Çift telli iletişim (SDA, SCL)	PB6, PB7
SPI	Dört telli yüksek hızlı iletişim	PA5, PA6, PA7
Güç	Besleme ve şase	3V3, 5V, GND
SWD	Debug ve programlama	PA13, PA14
Boot / reset	Sistem başlatma	BOOT0, NRST

1.2 GPIO modları

OUTPUT_PP	Çıkış, push-pull — HIGH/LOW verir	ANALOG	Analog okuma (ADC)
OUTPUT_OD	Çıkış, open-drain — pull-up ile kullanılır	IT_RISING / IT_FALLING	Giriş + kesme üretir (EXTI)
INPUT	Giriş — pin durumunu okur	AF_PP / AF_OD	Alternate function (UART, TIM vb.)

PUSH-PULL (PP)

HIGH verildiğinde pin 3,3 V'a, LOW verildiğinde 0 V'a sürülür. Her iki seviye de aktif olarak üretilir.

Çoğu durumda kullanılan varsayılan çıkış modu.

OPEN-DRAIN (OD)

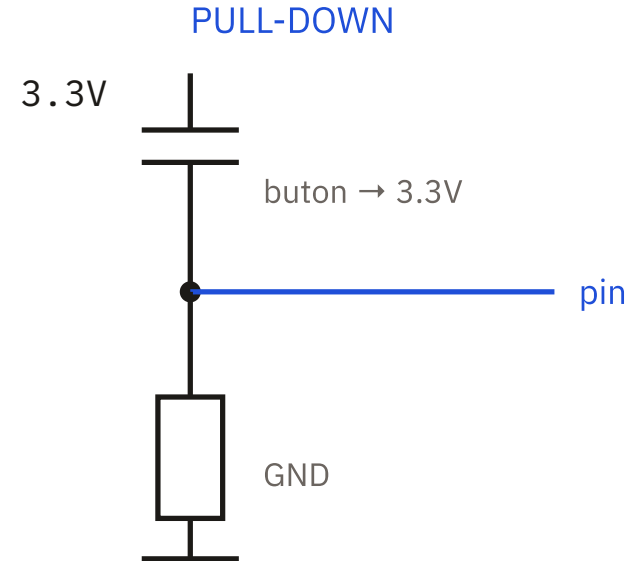
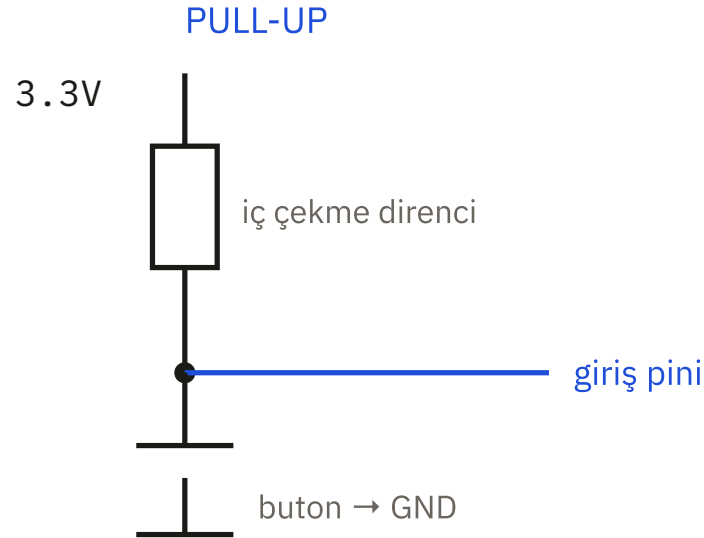
LOW verildiğinde pin 0 V'a çekilir. HIGH verildiğinde ise pin sürülmez, açık kalır — seviyeyi harici pull-up direnci belirler.

I2C gibi “wire-AND” protokollerinde zorunludur.

1.3 Pull-up ve pull-down nedir?

Bir giriş pini hiçbir yere bağlı değilse *float* durumdadır ve okunan değer belirsizdir; gürültüyle 0 veya 1 görünebilir. Pull-up direnci pini boştaiken 3,3 V'a, pull-down direnci ise 0 V'a çeker. Böylece “basılmamış” hâlin ne okunacağı garanti edilir.

Bu projede Hall pinlerinde GPIO_PULLUP var; boşta yüksek, sensör bastığında düşük.



ŞEKİL 2 — Pull-up'ta boşta seviye HIGH, buton LOW verir. Pull-down'da tersi geçerlidir.

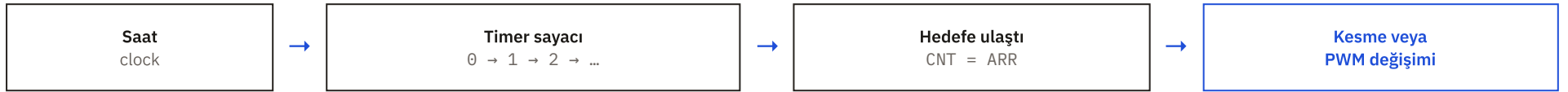
HIZLI TEKRAR · BÖLÜM 01

1. Bir pini UART için ayırmayı unutursanız program derlenir mi, çalışır mı?
2. Open-drain çıkış neden tek başına HIGH üretmez?
3. Pull-up'lı bir butonda basılmamış hâl hangi seviyedir?

02 Timer'lar

Timer, içinde bir sayaç (counter) bulunan çevre birimidir. Saat sinyalinden beslenir, belirli aralıklarla otomatik olarak sayar ve CPU'yu meşgul etmez. Sayaç hedef değere ulaştığında kesme üretir veya bir çıkış pininin seviyesini değiştirir.

Timer'ın değeri, işi **yazılım olmadan** yapmasıdır. CPU yalnızca sonucu okur.



ŞEKİL 3 — Timer'ın çalışma döngüsü.

2.1 Timer modları

Mod	Ne yapar	Örnek
Basic	Sadece sayar, kesme üretir	Her 1 s'de zamanlayıcı
PWM	Belirli oranda aç/kapa sinyali üretir	Motor hızı, LED parlaklığı
Encoder	A/B sinyallerini donanımda sayar	Motor pozisyon ve hızı
Input capture	Sinyalin değişim anını/süresini ölçer	Frekans ölçümü
One-pulse	Tetiklenince tek darbe üretir	Darbe üretme
Watchdog	Program takılırsa sistemi sıfırlar	Güvenlik

2.2 G4 ailesindeki timer'lar

Timer	Özellikler	Kullanım alanı
TIM1, TIM8, TIM20	Gelişmiş, 16-bit, PWM + komütasyon + encoder	Motor kontrolü (BLDC)
TIM2 ... TIM5	Genel amaçlı, 32/16-bit, encoder + PWM	Pozisyon ve PWM
TIM6, TIM7	Basic, 16-bit, yalnızca zamanlama	Periyodik kesme
LPTIM	Düşük güç timer'ı	Uyku modunda zaman
SysTick	ARM çekirdeğinin zamanlayıcısı	ms sayacı (HAL_GetTick)

2.3 PWM ve duty cycle

PWM (Pulse Width Modulation), bir pini hızla açıp kapatarak ortalama olarak “orta seviye” bir güç üretmektir. Duty cycle, bir periyot içinde sinyalin ne kadar süre açık kaldığının yüzdesidir. Frekans sabit tutulur, yalnızca açık kalma oranı değişir; motor hızı veya LED parlaklığı bu oranla ayarlanır.

Periyot = 1 / frekans. Duty cycle periyodu değiştirmez, yalnızca içindeki açık süreyi değiştirir.

100%



tam güç

75%



parlak

50%



yarı parlak

25%



soluk

0%



kapalı



bir periyot — bütün satırlarda aynı

ŞEKİL 4 — Aynı frekans, farklı duty cycle. Ortalama gerilim açık kalma oranıyla değişir.

BU PROJEDE

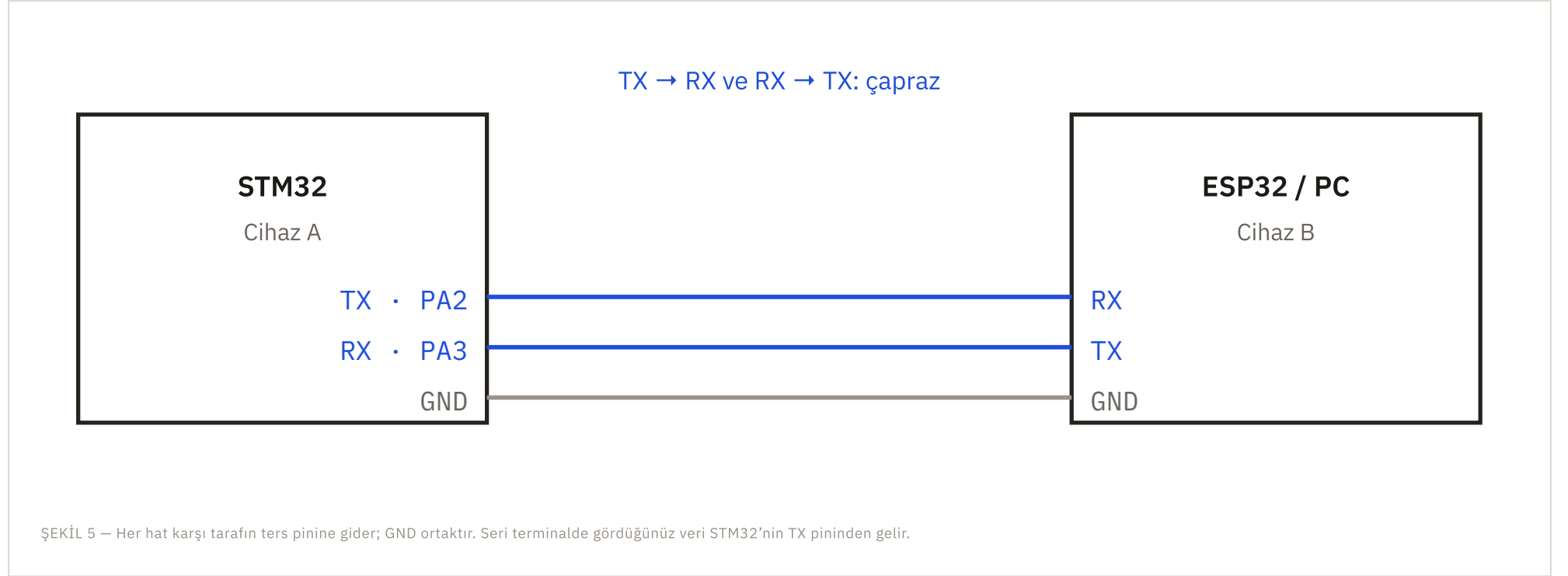
Motor PWM ile değil, GPIO aç/kapa ile sürülüyor: her 6-step adımında ilgili pinler elle HIGH/LOW yapılıyor. Gerçek motor kontrolünde ise sürücüye PWM verilir ve duty cycle doğrudan motor hızını belirler.

03 RX / TX pinleri: UART

UART (Universal Asynchronous Receiver/Transmitter), iki cihaz arasında iki tel üzerinden seri veri taşır. TX pini veri gönderir, RX pini veri alır. En yaygın seri iletişim yöntemidir ve ayrı bir saat teli gerektirmez — bu yüzden “asenكرون”dur.

Bağlantının tek kritik kuralı çaprazlamadır: A cihazının TX’i, B cihazının RX’ine gider. Ayrıca iki taraf ortak bir GND paylaşmak zorundadır.

Ortak GND olmadan seviyeler referanssız kalır ve iletişim çalışmaz. Bu **şarttır**, tercih değil.



3.1 Baud rate

Baud, saniyede gönderilen bit sayısıdır. Alıcı ve verici *aynı* baud ile çalışmak zorundadır; aksi hâlde veri anlamsız karakterlere dönüşür.

115200 → 115.200 bit/s

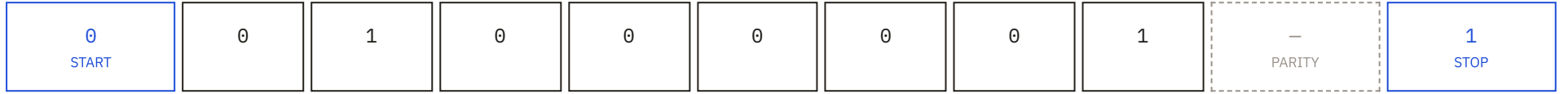
9600 → 9.600 bit/s

BU PROJEDE

115200 baud kullanılıyor. Düşük baud değerleri daha yavaştır, ancak uzun ve gürültülü hatlarda daha güvenilirdir.

3.2 Veri çerçevesi: 8N1

Bir karakter, örneğin 'A' = 0x41, hat üzerine şu sırayla çıkar: bir start biti, sekiz veri biti, isteğe bağlı parity biti ve bir ya da iki stop biti.



ŞEKİL 6 – 'A' = 0x41 = 0100 0001. Ortadaki sekiz kutu veri bitleridir.

WordLength 8 · Parity None · StopBits 1 → 8N1

3.3 Gönderme fonksiyonları

```
// Senkron – gönderme bitene kadar bekler (bloke edici)
HAL_UART_Transmit(&hlpuart1, data, size, timeout);

// Kesme tabanlı – göndermeye baslar, arka planda tamamlar
HAL_UART_Transmit_IT(&hlpuart1, data, size);

// DMA tabanlı – en verimli, büyük veriler için
HAL_UART_Transmit_DMA(&hlpuart1, data, size);
```


NEDEN LPUART1?

Bu projede USART yerine LPUART1 (Low Power UART) kullanılıyor. Aynı işi yapar, fakat düşük güç tüketir; PA2 = TX, PA3 = RX pinlerine bağlıdır. Düşük güç mimarisi nedeniyle üst baud sınırı daha düşüktür, 115200 civarında kalır.

HIZLI TEKRAR · BÖLÜM 03

1. TX-TX bağlarsanız ne olur?
2. Bir taraf 9600, diğeri 115200 baud ile çalışırsa terminalde ne görürsünüz?
3. 8N1 çerçevesinde bir karakter için hat üzerinde toplam kaç bit gider?

04 Hertz: frekans, periyot ve saat

Hertz, bir olayın saniyede kaç kez tekrarlandığını gösterir. Frekans tekrar sayısıdır, periyot ise tek bir tekrarın süresidir; ikisi birbirinin tersidir. Frekans büyüdükçe periyot küçülür.

Periyot = 1 / frekans. Zamanlama hesaplarında hemen her adım bu iki satır arasında gidip gelir.

1 Hz

periyot 1 s

10 Hz

periyot 100 ms

100 Hz

periyot 10 ms

170 MHz

periyot 5,88 ns

4.1 CPU saati ve dağıtımı

STM32 hızlı çalışabilmek için bir saat sinyali kullanır. Bu sinyal çekirdekten çevre birimlerine kadar bütün bloklara dağıtılır; her blok kendi bus'ı üzerinden beslenir.

SYSCCLK
170 MHz

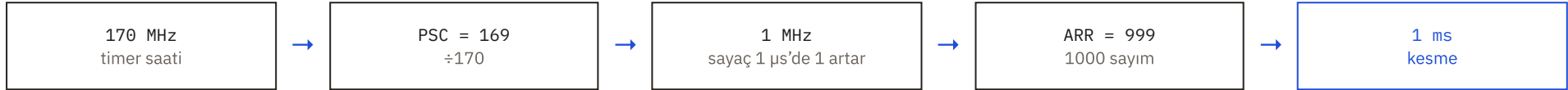
→ CPU — komutları çalıştırır	170 MHz
→ AHB bus — GPIO ve bellek	170 MHz
→ APB1 bus — timer'lar buna bağlı	bölünmüş
→ APB2 bus — UART, ADC, TIM	bölünmüş

ŞEKİL 7 — Saat dağıtımı. Bir çevre birimi çalışmıyorsa ilk kontrol edilecek şey saatinin açık olup olmadığıdır.

4.2 Timer'da Hz: prescaler ve auto-reload

Timer sayacı, timer saatinden beslenir; saat ne kadar hızlıysa sayaç o kadar hızlı artar. Prescaler (PSC) gelen frekansı böler, auto-reload (ARR) ise sayacın hangi değerde sıfırlanacağını belirler. İkisi birlikte kesme aralığını verir.

PSC ve ARR register'larına yazılan değer, istenen bölme oranından **bir eksiktir**: 170 bölme için PSC = 169.



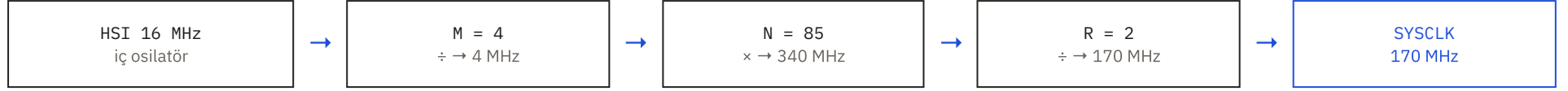
ŞEKİL 8 — 1 ms periyodik kesme: $1000 \times 1 \mu s = 1 ms$.

KESME FREKANSI FORMÜLÜ

$$f_{kesme} = f_{timer} / ((PSC + 1) \times (ARR + 1))$$

4.3 PLL ve saat kaynakları

CPU 170 MHz'e iç ya da dış osilatörden gelen düşük frekansı PLL ile çarparak ulaşır. PLL içinde önce bir bölme (M), sonra bir çarpma (N), sonra tekrar bir bölme (R) yapılır.



ŞEKİL 9 – PLL zinciri. Kaynak olarak HSI (16 MHz, iç) veya HSE (8/25 MHz, dış kristal) seçilebilir.

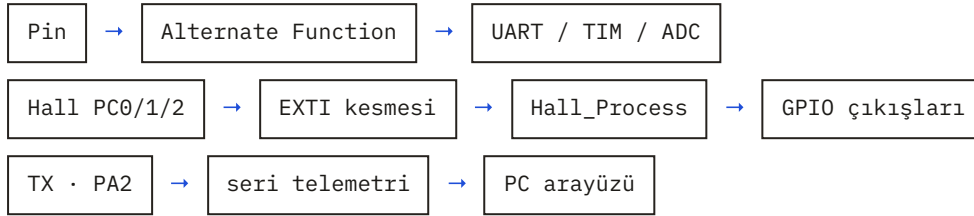
4.4 Bu projedeki zaman büyüklükleri

Kavram	Anlamı	Bu projede
SYSCLK	İşlemci saat hızı	170 MHz
HAL_GetTick()	SysTick'in saydığı milisaniye	Zaman ölçümü (now_ms)
UART baud	Seri veri hızı	115200 baud
Elektriksel RPM	Elektriksel dönüş hızı	6 adım × komütasyon frekansı
Frekans	Saniyedeki tekrar sayısı	Hall geçişleri

05 Her şey nasıl birbirine bağlanıyor?

CLOCK
170 MHz

- **CPU** komutları çalıştırır
- **Timer** sayaç, PWM, encoder arayüzü
- **GPIO** pinleri okur ve sürer
- **UART** veri gönderir (baud = bit/s)



ŞEKİL 10 — Saat her şeyi besler; pin fonksiyonu çevre birimini seçer; kesme yazılım akışını tetikler.

ANAHTAR TERİMLER

Alternate function	Bir pinin GPIO dışında bir çevre birimine (UART, TIM, ADC) atanması.
Push-pull / open-drain	Çıkışın her iki seviyeyi de sürmesi / yalnızca LOW'a çekmesi.
Float	Hiçbir yere bağlı olmayan, okunan değeri belirsiz giriş pini durumu.
Duty cycle	Bir PWM periyodunda sinyalin açık kaldığı sürenin yüzdesi.
PSC / ARR	Timer'ın prescaler ve auto-reload register'ları; birlikte sayma periyodunu belirler.
Baud rate	UART'ta saniyede taşınan bit sayısı; iki tarafta aynı olmak zorundadır.

8N1	8 veri biti, parity yok, 1 stop biti — en yaygın UART çerçeve biçimi.
SYSCCLK	Sistem saat frekansı; CPU ve bus'lar buradan beslenir.
PLL	Düşük frekanslı osilatör sinyalini bölüp çarparak yüksek sistem saati üreten blok.

Not: Pin alternatif fonksiyon numaraları, timer genişlikleri, bus bölme oranları ve PLL sınırları kullanılan STM32 modelinin datasheet ve reference manual belgelerinden doğrulanmalıdır.